Chapter 12

Cryptographic Hash Functions

MD5

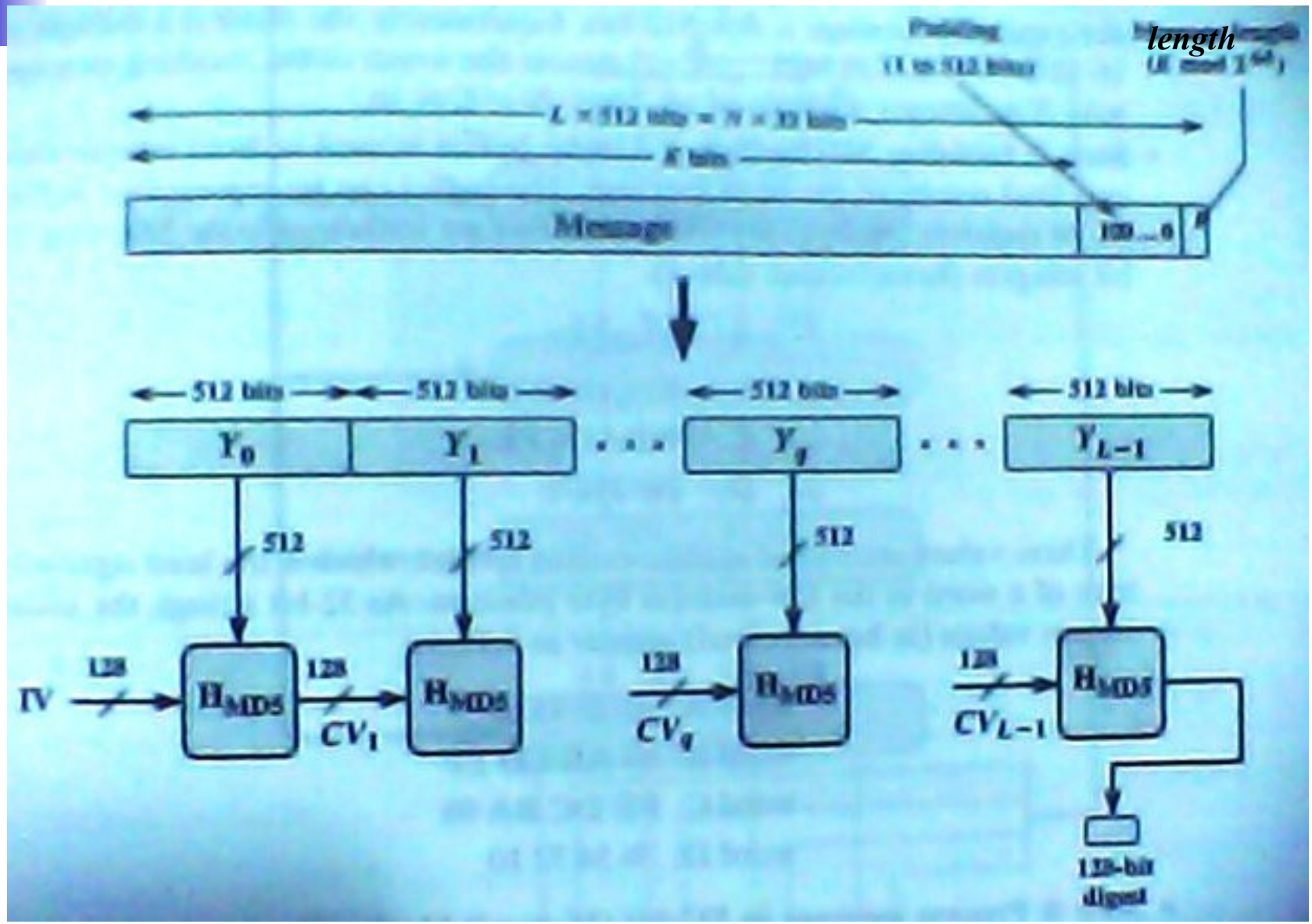
- *MD5 was developed by Professor Ronald L. Rivest of MIT.*

- *The MD5 algorithm] takes as input a message of arbitrary length and produces as output a 128-bit "fingerprint" or "message digest" of the input.*

- *The input is processed in 512 bit blocks.*

- *MD6 – the new version*
  - *Published on 15/04/2009*
  - *By Ronald L. Rivest*

MD5 Algorithm Description



Steps:

- The following five steps are performed to compute the message digest of the message.
- Step 1. Append Padding Bits
- Step 2. Append Length
- Step 3. Initialize MD Buffer
- Step 4. Process Message in 16-Word Blocks
- Step 5. Output

Step-1

- Append padding bits
  - The message is padded so that the length in bits is congruent to 448 mod 512
  - i.e. Length = 448 mod 512
  - Padding consists of a 1-bit followed by necessary no of zeroes.

12.2.1 Continued

$$|M| + |P| + 64 = 0 \text{ MOD } 512$$
$$|P| = [-M - 64] \text{ MOD } 512$$

Example 12.3

What is the number of padding bits if the length of the original message is 2590 bits?

Solution

We can calculate the number of padding bits as follows:

$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

The padding consists of one 1 followed by 353 0's.

Example 12.4

Do we need padding if the length of the original message is already a multiple of 1024 bits?

Solution

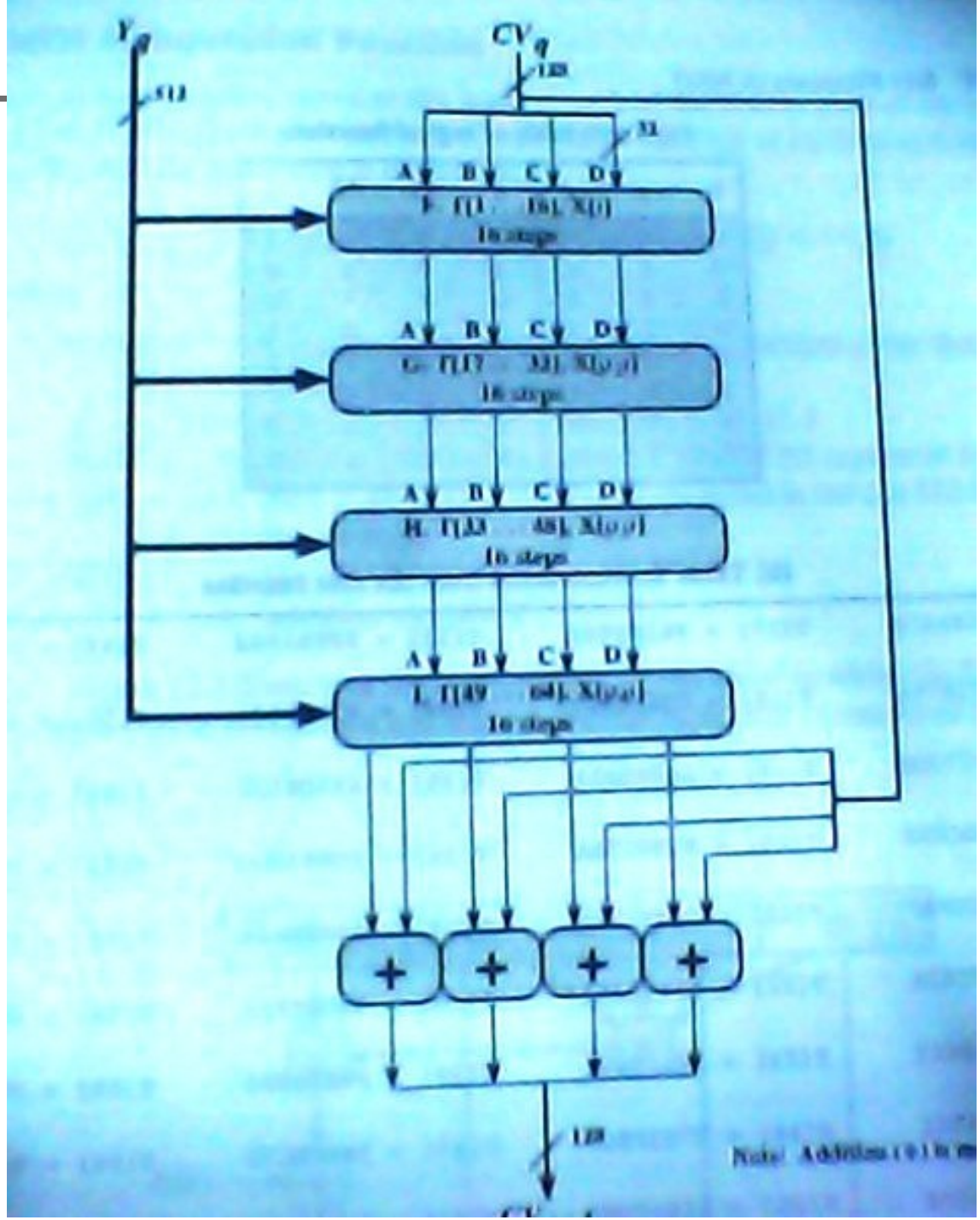
Yes we do, because we need to add the length field. So padding is needed to make the new block a multiple of 1024 bits.

STEP 3

- The main MD5 algorithm operates on a 128-bit state, divided into four 32-bit words,
- denoted *A*, *B*, *C* and *D*.
- These are initialized to certain fixed constants
- In little-endian format
  - i.e. least significant byte first
  - Ex: if A =87654321
    - The word A = 21436587

Step 4

- MD5 is a compression algorithm
- $CV_0 = IV$
- $CV_{q+1} = SUM_{32}[CVq, RF_I(Yq, RF_H(Yq, RF_G(Yq, RF_F(Yq, CVq))))]$
- $MD = CV_{L-1}$


A
B
C
D
+
+
+
+

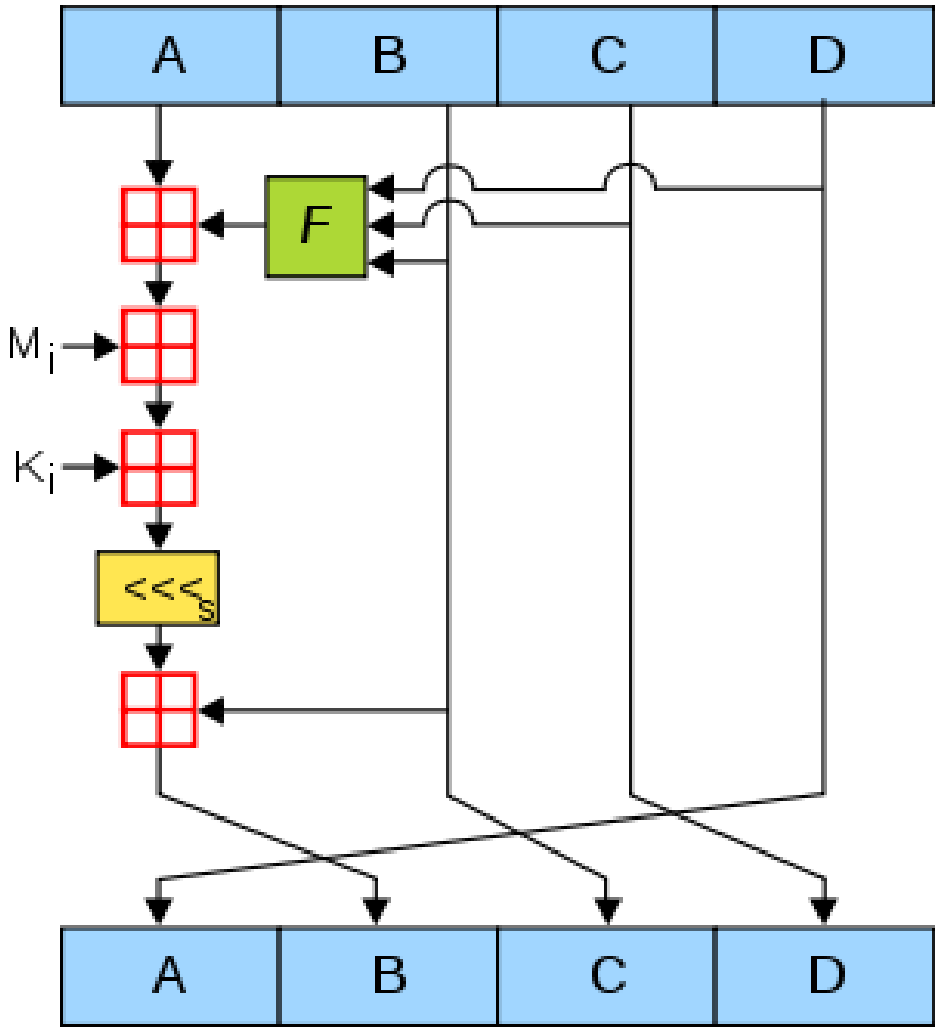
MD5 operations:

$$F(X,Y,Z) = (X \wedge Y) \vee (\neg X \wedge Z)$$

$$G(X,Y,Z) = (X \wedge Z) \vee (Y \wedge \neg Z)$$

$$H(X,Y,Z) = X \oplus Y \oplus Z$$

$$I(X,Y,Z) = Y \oplus (X \vee \neg Z)$$



12.1.2 Two Groups of Compression Functions

1. The compression function is made from scratch.

Message Digest (MD)

2. A symmetric-key block cipher serves as a compression function.

Whirlpool

12-1 INTRODUCTION

*A cryptographic hash function takes a message of arbitrary length and creates a message digest of fixed length. The ultimate goal of this chapter is to discuss the details of the two most promising cryptographic hash algorithms—**SHA-512 and Whirlpool.***

Topics discussed in this section:

12.1.1 Iterated Hash Function

12.1.2 Two Groups of Compression Functions

12.1.1 Iterated Hash Function

Merkle-Damgard Scheme

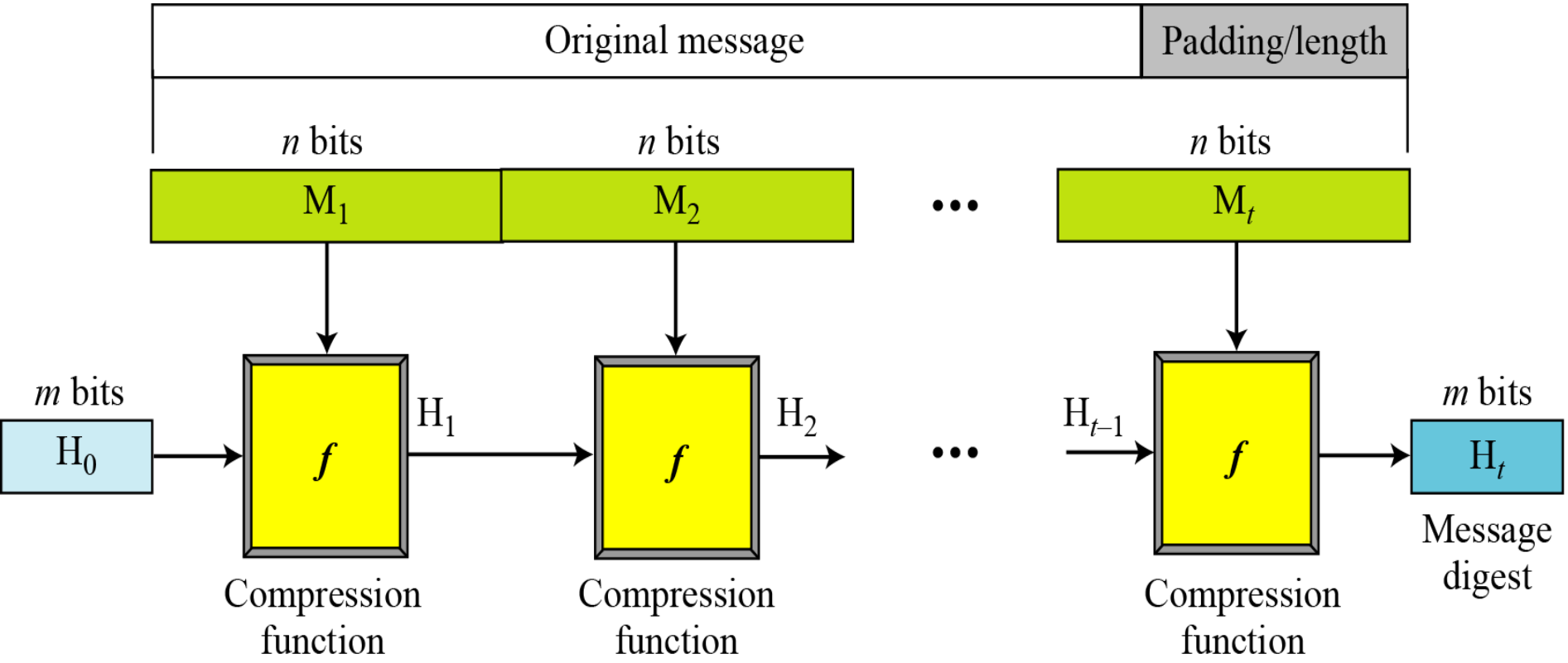


Figure 12.1 *Merkle-Damgard scheme*

12.1.2 Continued

Table 12.1 *Characteristics of Secure Hash Algorithms (SHAs)*

Characteristics	*SHA-1*	*SHA-224*	*SHA-256*	*SHA-384*	*SHA-512*
Maximum Message size	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size	512	512	512	1024	1024
Message digest size	160	224	256	384	512
Number of rounds	80	64	64	80	80
Word size	32	32	32	64	64

12-2 SHA-512

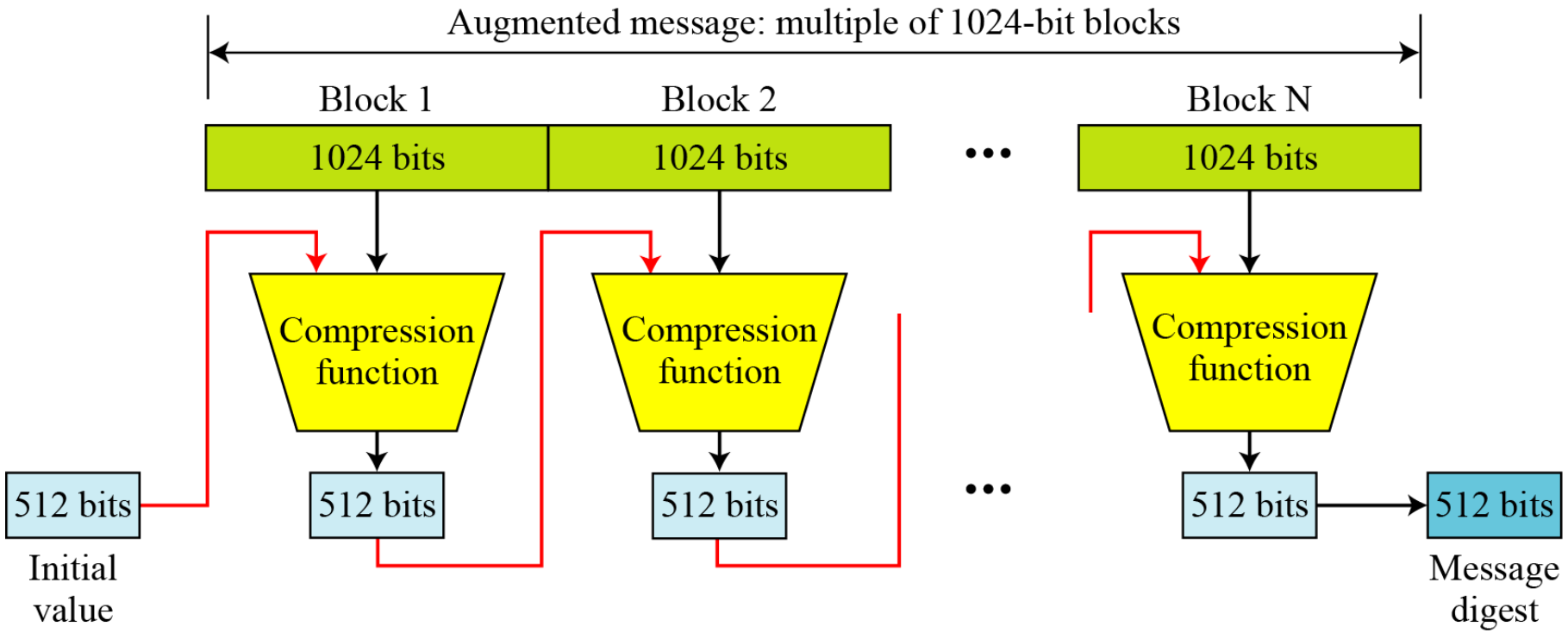
SHA-512 is the version of SHA with a 512-bit message digest. This version, like the others in the SHA family of algorithms, is based on the Merkle-Damgard scheme.

Topics discussed in this section:

12.2.1 Introduction
12.2.2 Compression Function
12.2.3 Analysis

12.2.1 Introduction

Figure 12.6 *Message digest creation SHA-512*



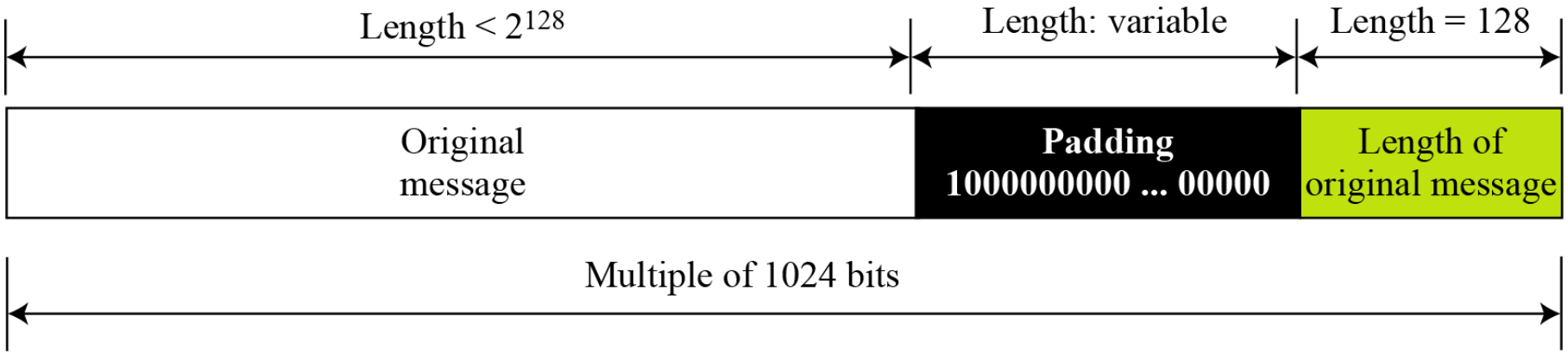
Message Preparation

SHA-512 insists that the length of the original message be less than 2^{128} bits.

Note

SHA-512 creates a 512-bit message digest out of a message less than 2^{128}.

Figure 12.7 *Padding and length field in SHA-512*



Example 12.5

What is the minimum and maximum number of padding bits that can be added to a message?

Solution

a. The minimum length of padding is 0 and it happens when $(-M - 128) \bmod 1024$ is 0. This means that $|M| = -128 \bmod 1024 = 896 \bmod 1024$ bits. In other words, the last block in the original message is 896 bits. We add a 128-bit length field to make the block complete.

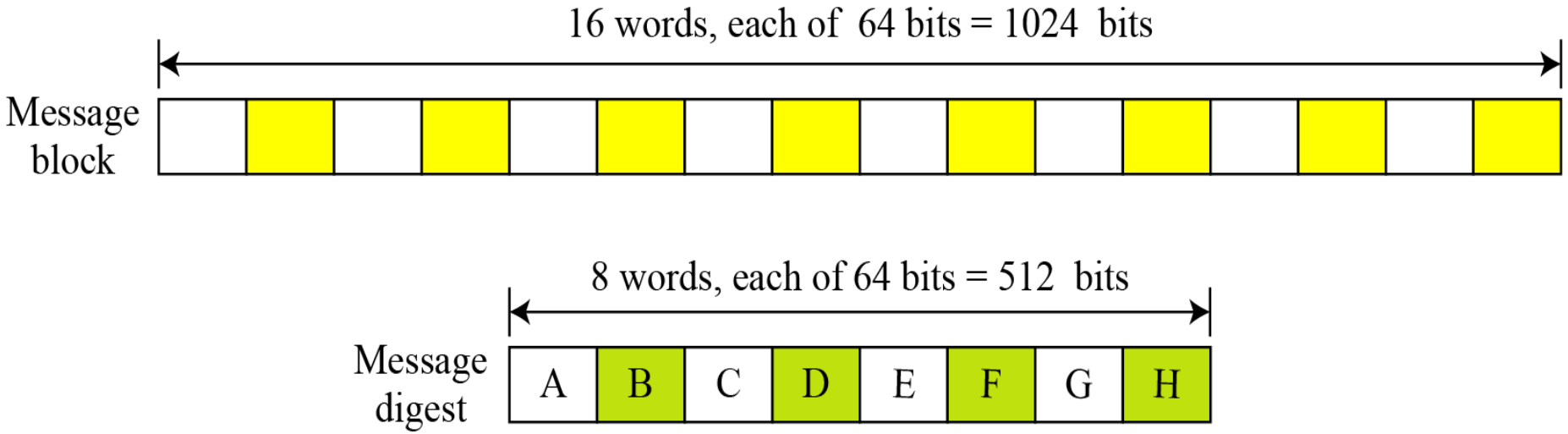
Example 12.5 *Continued*

b) The maximum length of padding is 1023 and it happens when $(-|M| - 128) = 1023 \bmod 1024$.
This means that the length of the original message is $|M| = (-128 - 1023) \bmod 1024$ or the length is $|M| = 897 \bmod 1024$.

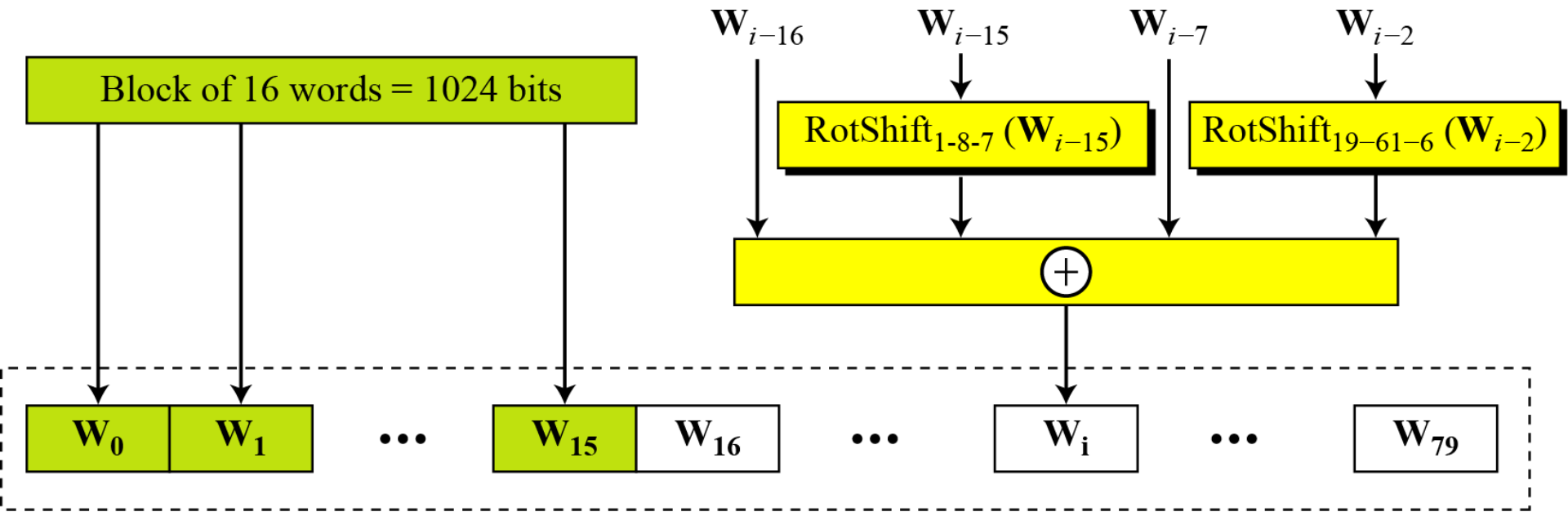
In this case, we cannot just add the length field because the length of the last block exceeds one bit more than 1024. So we need to add 897 bits to complete this block and create a second block of 896 bits. Now the length can be added to make this block complete.

Words

Figure 12.8 *A message block and the digest as words*



Word Expansion



$RotShift_{l\text{-}m\text{-}n}(x)$: $RotR_l(x) \oplus RotR_m(x) \oplus ShL_n(x)$

$RotR_i(x)$: Right-rotation of the argument x by i bits
$ShL_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

Figure 12.9 *Word expansion in SHA-512*

Example 12.6

Show how W60 is made.

Solution

Each word in the range W16 to W79 is made from four previously-made words. W60 is made as

$$W_{60} = W_{44} \oplus \text{RotShift}_{1\text{-}8\text{-}7}(W_{45}) \oplus W_{53} \oplus \text{RotShift}_{19\text{-}61\text{-}6}(W_{58})$$

Message Digest Initialization

Table 12.2 *Values of constants in message digest initialization of SHA-512*

Buffer	*Value (in hexadecimal)*	*Buffer*	*Value (in hexadecimal)*
A_0	6A09E667F3BCC908	E_0	510E527FADE682D1
B_0	BB67AE8584CAA73B	F_0	9B05688C2B3E6C1F
C_0	3C6EF372EF94F828	G_0	1F83D9ABFB41BD6B
D_0	A54FE53A5F1D36F1	H_0	5BE0CD19137E2179

12.2.2 Compression Function

Figure 12.10 *Com*

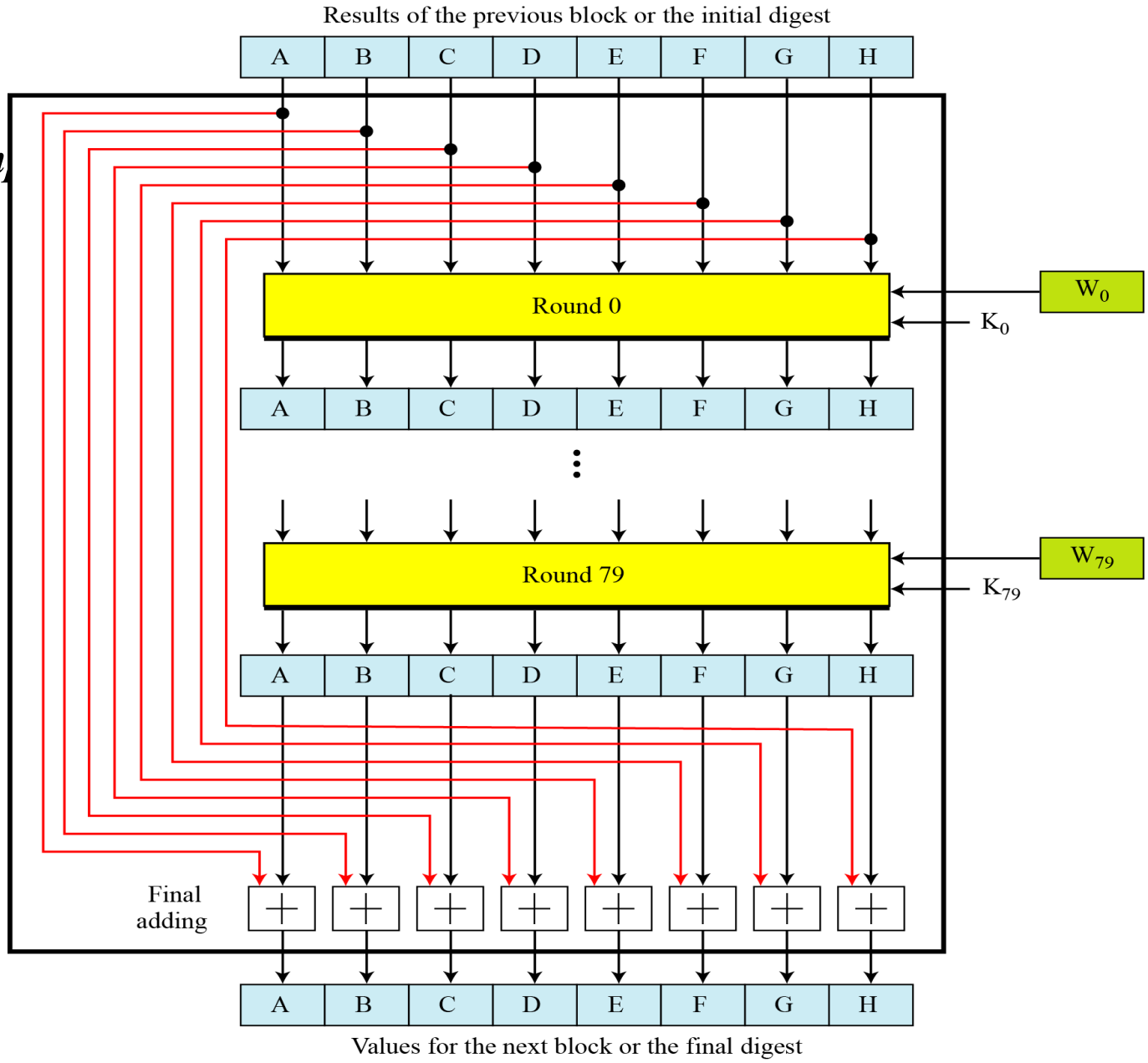
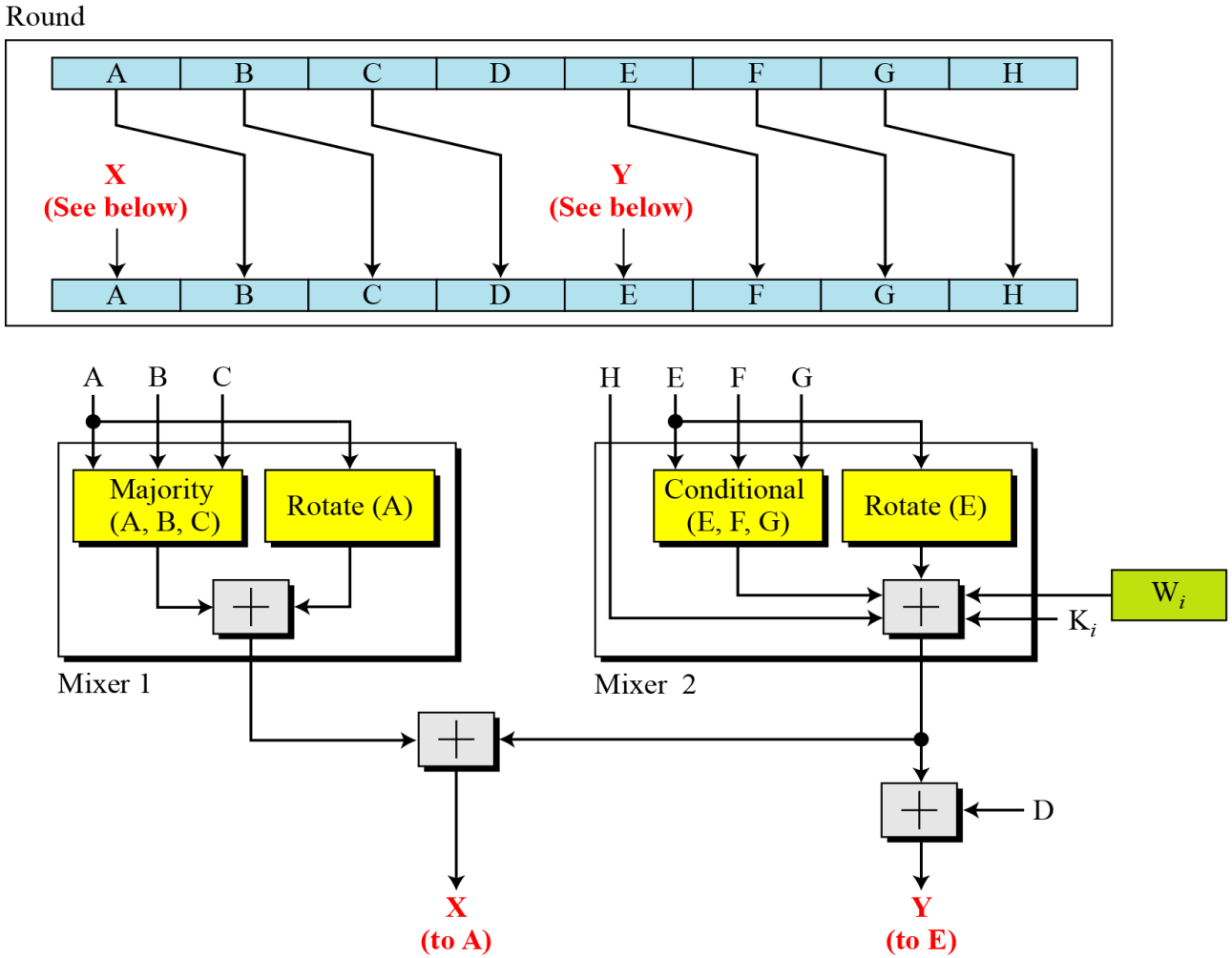


Figure 12.11 *Structure of each round in SHA-512*



12.2.2 Continued

Majority Function

$$(A_j \text{ AND } B_j) \oplus (B_j \text{ AND } C_j) \oplus (C_j \text{ AND } A_j)$$

Conditional Function

$$(E_j \text{ AND } F_j) \oplus (\text{NOT } E_j \text{ AND } G_j)$$

Rotate Functions

12.2.2 Continued

Table 12.3 *Eighty constants used for eighty rounds in SHA-512*

428A2F98D728AE22	7137449123EF65CD	B5C0FBCFEC4D3B2F	E9B5DBA58189DBBC
3956C25BF348B538	59F111F1B605D019	923F82A4AF194F9B	AB1C5ED5DA6D8118
D807AA98A3030242	12835B0145706FBE	243185BE4EE4B28C	550C7DC3D5FFB4E2
72BE5D74F27B896F	80DEB1FE3B1696B1	9BDC06A725C71235	C19BF174CF692694
E49B69C19EF14AD2	EFBE4786384F25E3	0FC19DC68B8CD5B5	240CA1CC77AC9C65
2DE92C6F592B0275	4A7484AA6EA6E483	5CB0A9DCBD41FBD4	76F988DA831153B5
983E5152EE66DFAB	A831C66D2DB43210	B00327C898FB213F	BF597FC7BEEF0EE4
C6E00BF33DA88FC2	D5A79147930AA725	06CA6351E003826F	142929670A0E6E70
27B70A8546D22FFC	2E1B21385C26C926	4D2C6DFC5AC42AED	53380D139D95B3DF
650A73548BAF63DE	766A0ABB3C77B2A8	81C2C92E47EDAEE6	92722C851482353B
A2BFE8A14CF10364	A81A664BBC423001	C24B8B70D0F89791	C76C51A30654BE30
D192E819D6EF5218	D69906245565A910	F40E35855771202A	106AA07032BBD1B8
19A4C116B8D2D0C8	1E376C085141AB53	2748774CDF8EEB99	34B0BCB5E19B48A8
391C0CB3C5C95A63	4ED8AA4AE3418ACB	5B9CCA4F7763E373	682E6FF3D6B2B8A3
748F82EE5DEFB2FC	78A5636F43172F60	84C87814A1F0AB72	8CC702081A6439EC
90BEFFFA23631E28	A4506CEBDE82BDE9	BEF9A3F7B2C67915	C67178F2E372532B
CA273ECEEA26619C	D186B8C721C0C207	EADA7DD6CDE0EB1E	F57D4F7FEE6ED178
06F067AA72176FBA	0A637DC5A2C898A6	113F9804BEF90DAE	1B710B35131C471B
28DB77F523047D84	32CAAB7B40C72493	3C9EBE0A15C9BEBC	431D67C49C100D4C
4CC5D4BECB3E42B6	4597F299CFC657E2	5FCB6FAB3AD6FAEC	6C44198C4A475817

12.2.2 Continued

There are 80 constants, K_0 to K_{79}, each of 64 bits. Similar These values are calculated from the first 80 prime numbers (2, 3,…, 409). For example, the 80th prime is 409, with the cubic root $(409)^{1/3} = 7.4229412044$. Converting this number to binary with only 64 bits in the fraction part, we get

$$(111.0110\ 1100\ 0100\ 0100\ \ldots\ 0111)_2 \rightarrow (7.6C44198C4A475817)_{16}$$

The fraction part: $(6C44198C4A475817)_{16}$

Example 12.7

We apply the Majority function on buffers A, B, and C. If the leftmost hexadecimal digits of these buffers are 0x7, 0xA, and 0xE, respectively, what is the leftmost digit of the result?

Solution

The digits in binary are 0111, 1010, and 1110.

a. The first bits are 0, 1, and 1. The majority is 1.

b. The second bits are 1, 0, and 1. The majority is 1.

c. The third bits are 1, 1, and 1. The majority is 1.

d. The fourth bits are 1, 0, and 0. The majority is 0.

The result is 1110, or 0xE in hexadecimal.

Example 12.8

We apply the Conditional function on E, F, and G buffers. If the leftmost hexadecimal digits of these buffers are 0x9, 0xA, and 0xF respectively, what is the leftmost digit of the result?

Solution

The digits in binary are 1001, 1010, and 1111.

a. The first bits are 1, 1, and 1. The result is F_1, which is 1.

b. The second bits are 0, 0, and 1. The result is G_2, which is 1.

c. The third bits are 0, 1, and 1. The result is G_3, which is 1.

d. The fourth bits are 1, 0, and 1. The result is F_4, which is 0.

The result is 1110, or 0xE in hexadecimal.